

TFG Identification Module

for Embention KAI — Aerial Object Identification System

Author	Taron Sargsyan
Institution	University of Alicante — AI Engineering
Partner	Embention — Vision Department (KAI Platform)
Target domain	Aerial objects: UAV drones, fixed-wing, helicopters
Op. distance	50–100 m (2×1 m target class)
SDK (Linux)	11-01-00-03 → edgeai-tidl-tools tag: 11_01_xx
SDK (RTOS)	11_01_00_04 (baremetal, out of TFG scope)
Date	June 2026
Version	v2 — Sky domain, Option A privacy, Approach 2 viewpoint

Changes from v1	
Domain	Ground vehicles → aerial objects (drones, planes, helicopters)
Privacy	Option A: client-side embedding, only prototype vector transmitted
Viewpoint	Approach 2: attention-weighted aggregation over N support views
Resolution	Sensor 1920×1080, HFOV 30–45°. Degradation pipeline configurable, not mandatory; necessity determined by hardware FPS trade-off.
Quantization	Both FP32 and INT8 ONNX exports as Hito 3 deliverable
N (shots)	Determined empirically via ablation study; preliminary target: 5 images from different angles

Table of Contents

Executive Summary

- E.1 What this document covers
- E.2 Legal and ethical framing

1. Project Overview

- 1.1 Problem definition
- 1.2 Scope boundaries
- 1.3 Success criteria

2. YOLO Crops, Resolution & the Domain Gap

- 2.1 Pipeline input: YOLO crops only
- 2.2 Pixel math at operational distances (1920×1080)
- 2.3 Domain gap: reference images vs YOLO crops
- 2.4 Resolution-matched enrollment — configurable option

3. System Architecture

- 3.1 End-to-end pipeline overview
- 3.2 Two ONNX models: backbone and attention head
- 3.3 Gallery structure and transmission

4. Privacy Architecture — Option A

- 4.1 Design principle: images never leave client
- 4.2 Client application architecture
- 4.3 Transmission security and threat model
- 4.4 Client app code sketch

5. Viewpoint-Aware Embedding — Approach 2

- 5.1 Why naive mean prototype fails at N=5 across angles
- 5.2 Attention-weighted aggregation architecture
- 5.3 Joint training strategy
- 5.4 Runtime inference flow

6. Model Architecture

- 6.1 Backbone: MobileNetV3-Small
- 6.2 Attention head: lightweight MLP cross-attention
- 6.3 ONNX export for both models

7. Dataset Strategy — UAV Domain

- 7.1 Available public datasets (UAV-only)
- 7.2 UAV reference images for the ablation study
- 7.3 Augmentation pipeline for UAV domain

8. Quantization Pipeline

- 8.1 Why INT8 matters for edge hardware
- 8.2 Post-training quantization flow
- 8.3 What gets quantized and what does not
- 8.4 Hito 3 deliverable specification

9. N-shot Ablation — A Product Question

- 9.1 Reframing: this is product specification research

9.2 Test matrix: N and image type

9.3 Expected output: practical recommendation

10. Risks and Mitigations

11. Open Questions for Embention

12. Development Roadmap — 12 Weeks

Month 1: Foundation (Weeks 1–4)

Month 2: Attention module and ablation (Weeks 5–8)

Month 3: Optimisation and demo (Weeks 9–12)

Executive Summary

This document is version 2 of the technical analysis for the TFG Identification Module developed in collaboration with Embention. The project scope has shifted significantly from the initial ground vehicle detection brief. The new objective is an aerial object identification system: a third-party client uploads a small number of reference images of a target aerial object (a specific drone model, aircraft, or helicopter), and the drone memorises that object so that when it encounters it at operational distances of 50 to 100 metres, it can report a positive identification to the Autonomous Target Engagement (ATE) system.

Three major architectural decisions have been locked in this version. First, privacy is implemented via **Option A**: the client application runs the backbone model locally on the client's own hardware, and only a compact embedding vector (512 bytes) is transmitted to the drone — the reference images never reach Embention's servers or any external network. Second, viewpoint variation across the N reference images is handled via **Approach 2**: an attention-weighted aggregation head that dynamically computes a query-conditioned prototype at runtime, rather than a static mean of all support embeddings. Third, the pipeline produces both **FP32 and INT8 ONNX exports** as hardware-agnostic deliverables that Embention's hardware team can drop into any target platform compiler without changes to the model.

E.1 What this document covers

Sections 1 through 9 cover the technical design in full detail: the resolution challenge at operational distances, the end-to-end pipeline architecture, the privacy model, the attention-based viewpoint aggregation, the dataset strategy for the sky domain, and the quantization pipeline. Section 10 lists updated risks and mitigations for the new scope. Section 11 lists open questions that must be resolved with Embention. Section 12 provides the full 12-week development roadmap with updated week-by-week tasks and deliverables.

E.2 Legal and Ethical Framing

The identification module described in this document produces a single output: a confidence score indicating whether the detected aerial object matches the enrolled target. It does not issue engagement commands, does not control actuators, and does not make kinetic decisions. The downstream decision to engage — the physical action the drone takes — is governed by the Autonomous Target Engagement (ATE) system and is explicitly outside this TFG scope.

Legal requirement: Under current international humanitarian law and the emerging regulatory frameworks governing autonomous weapons systems, the engagement decision must remain under meaningful human control. The identification module's output is a decision-support signal, not an autonomous kill command. This framing must be preserved in all documentation, presentations, and communications with third-party clients. The TFG proposal and final report should explicitly state this boundary on the cover page and in the introduction.

For the TFG examination context, the project is framed as a vision and machine learning engineering challenge: privacy-preserving few-shot identification of aerial objects under real-world operational constraints. The ATE system it connects to is referenced as an integration point, not a deliverable.

1. Project Overview

1.1 Problem Definition

A third-party client (a military operator or defence contractor) possesses a target object they want their drone to identify in the sky. They have no access to large annotated datasets of that object. They have a small number of reference photographs — side views, front views, three-quarter views — taken from available sources. The requirement is: upload these images through a local application, and from that moment the drone can recognise the object if it appears within operational range.

The initial target class for the TFG proof of concept is **UAVs only** — consumer, commercial, and military unmanned aerial vehicles in the 2×1 metre size range. Planes and helicopters are explicitly out of scope for this first iteration. The system must generalise across different UAV types: the same pipeline should identify a DJI Mavic, a Bayraktar TB2, or any other enrolled drone without retraining the backbone.

The identification module receives object crops produced by a generic upstream detector (YOLOv8n). It does not process full frames and does not use sliding windows or patch-based strategies. Only the bounding box crops output by YOLO reach the identification backbone. This keeps the identification module architecturally clean and independent of the detector choice.

1.2 Scope Boundaries

In scope	Out of scope
• Few-shot identification module: Prototypical Network backbone + attention head.	• ATE engagement decision logic (separate Embention subsystem).
• Client application: local ONNX inference to produce prototype gallery without transmitting images.	• YOLO detector training (pre-trained model used off the shelf).
• Resolution-matched enrollment: degradation pipeline simulating 50–100 m appearance.	• Hardware-specific TIDL/TensorRT compilation (handled later by Embention hardware team).
• ONNX export pipeline: FP32 backbone, INT8 backbone, and FP32 attention head.	• Client application GUI (CLI script is the TFG deliverable; GUI is a later product step).
• N-shot ablation study: determining the optimal number and type of reference images.	• Encrypted RF communication protocol for gallery transmission (flagged as integration point).
• End-to-end demo: pre-trained YOLO detector + identification module on aerial video.	•
• Privacy threat model and documentation for Embention client-facing materials.	•

1.3 Success Criteria

Criterion	Definition
Viability demo	The system correctly identifies an enrolled aerial target from a live video stream using N=5 reference images, with similarity score above threshold, producing zero false positives on non-enrolled objects in the same footage.

Trade-off analysis	A documented accuracy vs N curve showing performance at N = 1, 3, 5, 10, 20, and a justified recommendation for the minimum N for operational use.
Privacy guarantee	The client application produces gallery.npy from reference images without any network call to Embention or any third party. Demonstrable by running with network monitoring (e.g. Wireshark).
ONNX exports	backbone_fp32.onnx, backbone_int8.onnx, and attention_head.onnx all load and produce correct outputs in standard onnxruntime on x86. Accuracy delta FP32 vs INT8 documented.
Code quality	Repository reproducible from a clean clone with a single training command. README covers installation, training, gallery generation, and demo usage. Another Embention engineer can replicate all results.

2. YOLO Crops, Resolution & the Domain Gap

2.1 Pipeline Input: YOLO Crops Only

The identification module does not process full 1920×1080 frames. The upstream YOLO detector runs on the full frame, produces bounding box detections, and the identification module receives only those crops — one 224×224 resized patch per detected object per frame. No sliding windows, no overlapping patches, no full-frame strategies. This is an explicit architectural decision: it keeps the identification module independent of the detector, makes it fast enough for real-time processing, and means the only input the backbone ever sees is a tight crop around the detected UAV.

Pipeline invariant: full 1920×1080 frame → YOLOv8n detector → bounding box crops → resize to 224×224 → backbone.onnx → attention_head.onnx → similarity score. The identification module is only ever called with a YOLO crop, never with a raw frame.

2.2 Pixel Math at Operational Distances

Understanding the pixel count at operational distance is critical because it determines how much information the backbone receives and whether the domain gap between clean reference images and runtime crops is severe enough to require a degradation pipeline during enrollment. The formula for object width in pixels on a 1920×1080 sensor is:

```
# pixel_width = sensor_px * object_width / (2 * distance * tan(HFOV/2))
# For 1920 x 1080, HFOV 30deg, 2m UAV at 100m:
# = 1920 * 2 / (2 * 100 * tan(15deg)) = 3840 / 53.6 = 71.6 px
```

Distance	HFOV 30° (wide × tall)	Upsample to 224	HFOV 45° (wide × tall)	Upsample to 224
50 m	143 × 73 px	1.6×	93 × 48 px	2.4×
75 m	96 × 49 px	2.3×	62 × 32 px	3.6×
100 m	72 × 37 px	3.1×	46 × 24 px	4.9×

Sensor: 1920×1080. Object: 2×1 m UAV. VFOV = HFOV × 0.5625 (square pixels). Object centred in frame.

At HFOV 30° and 50 m the crop is 143 px wide — nearly no information loss after upsampling to 224. At HFOV 45° and 100 m the crop is 46 px wide — blocky after 4.9× upsampling, but wing planform, rotor disc geometry, and body proportions are still structurally present. This is manageable: the backbone is not working from noise. However, the domain gap between a clean reference photo and a 46 px upsampled crop is real and must be addressed in the enrollment strategy.

2.3 Domain Gap: Reference Images vs YOLO Crops

The client uploads high-resolution reference photographs. The backbone computes embeddings from these clean images during enrollment. At runtime, the same backbone receives a 224×224 patch upsampled from a 46–143 px YOLO crop. These two inputs can occupy different regions of the feature space even when they depict the same object, because the backbone activates on texture and fine-grained edges that are present in the clean reference but absent after downsampling. Cosine similarity between a clean enrollment embedding and a degraded runtime embedding will be suppressed.

The gap is distance-dependent and hardware-dependent. At 50 m with HFOV 30° (143 px, 1.6× upsample) the gap may be negligible. At 100 m with HFOV 45° (46 px, 4.9× upsample) it is significant. Whether a degradation pipeline is needed in production depends on the FPS budget: if the hardware can afford a higher-resolution crop or a lighter backbone at the target framerate, the gap shrinks. This decision is deferred to hardware testing. For the TFG, the degradation pipeline is implemented as a configurable option and the ablation study measures whether it helps.

2.4 Resolution-Matched Enrollment — Configurable Option

The degradation pipeline in `generate_gallery.py` is off by default. When enabled via the `--match_distance` flag, it simulates the appearance of the YOLO crop at the specified distance before computing the embedding. The correct `target_px` values for the 1920×1080 sensor are derived from the table above. The ablation study in Week 6 includes a row comparing enrollment with and without degradation at each distance, directly measuring whether the pipeline is worth activating.

`generate_gallery.py` — configurable degradation

```
# generate_gallery.py — degradation as a configurable option
# target_px lookup for 1920x1080 sensor, 2m-wide UAV
DISTANCE_TABLE = {
    ("hfov30", "50m"): 143, # near-lossless
    ("hfov30", "75m"): 96,
    ("hfov30", "100m"): 72,
    ("hfov45", "50m"): 93,
    ("hfov45", "75m"): 62, # default if degradation enabled
    ("hfov45", "100m"): 46, # worst case: most aggressive
}

def degrade(img, target_px, jpeg_quality=60):
    """Simulate YOLO crop appearance at operational distance."""
    h, w = img.shape[:2]
    scale = target_px / max(h, w)
    small = cv2.resize(img, (max(1, int(w*scale)), max(1, int(h*scale))),
        interpolation=cv2.INTER_AREA)
    k = np.zeros((5,5)); k[2,:] = 0.2 # horizontal motion blur
    small = cv2.filter2D(small, -1, k)
    _, enc = cv2.imencode(".jpg", small, [cv2.IMWRITE_JPEG_QUALITY, jpeg_quality])
    return cv2.resize(cv2.imdecode(enc, 1), (224, 224))

def embed_with_optional_degradation(session, img, match_distance=None):
    """
    match_distance: None (no degradation, default) or
    e.g. ("hfov45", "100m") to simulate worst case.
    """
    if match_distance is not None:
        target_px = DISTANCE_TABLE[match_distance]
        img = degrade(img, target_px)
    else:
        img = cv2.resize(img, (224, 224)) # clean enrollment
    x = img.transpose(2,0,1)[None].astype("float32") / 255.0
    emb = session.run(None, {"image": x})[0][0]
    return emb / (np.linalg.norm(emb) + 1e-8)
```

During training, degradation is applied with random `target_px` sampled from $U(46, 143)$ — covering the full operational envelope from HFOV 45° at 100 m to HFOV 30° at 50 m. This trains the backbone to be robust across the entire distance range. With probability 0.3, no degradation is applied at all, ensuring the

backbone also performs well with clean high-resolution crops.

3. System Architecture

3.1 End-to-End Pipeline Overview

The system operates in two distinct phases: **enrollment** (happens once, pre-flight, on the client's machine) and **runtime** (happens continuously on the drone during the mission). These two phases must share the same backbone model weights and the same degradation parameters, but they run on different hardware and at different times.

	Enrollment (client machine)	Runtime (drone)
Where	Client's laptop or workstation	Onboard drone computer
When	Once, before each mission	Continuously during flight
Input	N reference images per target class	Live camera frames at 30 fps
Models used	backbone.onnx only	backbone.onnx + attention_head.onnx
Output	gallery.npy (N embeddings per class)	Similarity score per frame
Network	None (fully offline)	None (fully offline)
Data transmitted	gallery.npy only (AES-256 encrypted)	Nothing

3.2 Two ONNX Models

The system deliberately separates into two ONNX models. This separation is intentional: it keeps the backbone (the heavy model, ~2M parameters) shareable across enrollment and runtime, while keeping the attention head (the lightweight model, ~147K parameters) as a runtime-only component that the client app never needs.

Model	File	Params	Input	Used in
Feature backbone	backbone_fp32.onnx backbone_int8.onnx	~2.5 M	(1, 3, 224, 224)	Enrollment + Runtime
Attention head	attention_head.onnx	~147 K	query (128,) + supports (N, 128)	Runtime only

3.3 Gallery Structure and Transmission

The gallery file is a NumPy .npy dictionary mapping class names to arrays of shape (N, 128) — one 128-dimensional embedding per reference image. This is deliberately NOT a mean prototype. Storing all N embeddings individually is required by Approach 2: the attention head needs all N support vectors at runtime to compute query-conditioned weights. A gallery for a single enrolled target class with N=5 images contains 5 × 128 × 4 bytes = 2.5 KB of data. Encrypted with AES-256, this is a trivial transfer over USB, WiFi, or RF.

gallery.npy — structure and loading

```
# Gallery structure (gallery.npy content)
{
"MQ-9_Reaper": np.ndarray(shape=(5, 128), dtype=float32),
"TB2_Bayraktar": np.ndarray(shape=(5, 128), dtype=float32),
# ... one entry per enrolled target class
}
# Loading at runtime (on drone)
gallery = np.load("gallery.npy", allow_pickle=True).item()
supports = gallery["MQ-9_Reaper"] # shape (5, 128)
```

4. Privacy Architecture — Option A

4.1 Design Principle: Images Never Leave the Client

Option A is built around a single invariant: the client's reference images are processed exclusively on the client's own hardware. Embention distributes the backbone as a compiled ONNX file bundled with the client application. The application runs inference locally, applies the degradation pipeline, and produces gallery.npy. From Embention's perspective, the client application is a black box: they cannot see what images were uploaded, cannot infer the target from the gallery vectors, and have no network access to the client's machine during operation.

Privacy guarantee: The client application makes zero outbound network calls during enrollment. The only file that leaves the client's machine is gallery.npy, which contains floating-point embedding vectors. Reconstructing a recognisable image from a 128-dimensional embedding is computationally infeasible with current methods — the information bottleneck is by design.

4.2 Client Application Architecture

The client application is a Python package (later potentially wrapped in a simple GUI) that ships with the backbone ONNX file bundled inside it. The client installs it once and uses it pre-flight. The package has three dependencies visible to the client: onnxruntime, numpy, and opencv. No cloud SDK, no API key, no internet requirement.

Step	Where it runs	What happens	Data involved
1. Install	Client machine	pip install embention-enroll; includes backbone.onnx	backbone.onnx distributed once
2. Upload	Client machine	User places N images in a folder	Images stay local
3. Degraded	Client machine	degrade_reference() applied to each image	Degraded tensors in RAM only
4. Embed	Client machine	backbone.onnx produces 128-dim vector per image	Embeddings in RAM only
5. Save	Client machine	gallery.npy written to disk	2.5 KB per class, locally stored
6. Transfer	USB / local RF	AES-256 encrypted gallery.npy loaded onto drone	Ciphertext only
7. Fly	Drone	gallery.npy decrypted in RAM, runtime inference	Nothing leaves drone

4.3 Transmission Security and Threat Model

The gallery.npy file is encrypted with AES-256-GCM before leaving the client's machine. The encryption key is derived from a pre-shared secret between the client and the drone (exchanged during initial hardware configuration). This is standard military-grade key management and is out of TFG scope; the TFG deliverable flags this as a required integration point and documents the interface.

Threat	Asset at risk	Mitigation	In TFG scope
Attacker intercepts gallery.npy in transit	Target identity	AES-256-GCM encryption; key never in gallery file	Documented, not implemented
Attacker recovers images from embedding vectors	Reference images	128-dim bottleneck makes inversion infeasible	Yes — architecture provides this
Client reverse-engineers backbone.onnx	Embection model IP	ONNX obfuscation / hardware binding (future step)	Not in TFG scope
Drone captured; gallery.npy read from storage	Target identity	Encrypted at-rest storage; key in secure enclave	Documented, not implemented

4.4 Client App Code Sketch

The following is the core of the client application as a TFG deliverable. It is a clean, documented Python script that Embection can later wrap in a GUI or package as a standalone executable.

client_app/generate_gallery.py — complete script

```
# client_app/generate_gallery.py
# Runs exclusively on client machine. Zero network calls.
import onnxruntime as ort, numpy as np, cv2
from pathlib import Path
BACKBONE_PATH = "backbone_fp32.onnx" # bundled with app
TARGET_PX = 44 # simulates 75 m operational distance
EMBED_DIM = 128
def degrade(img, target_px=TARGET_PX, q=60):
    h, w = img.shape[:2]
    s = target_px / max(h, w)
    small = cv2.resize(img, (max(1,int(w*s)), max(1,int(h*s))),
        interpolation=cv2.INTER_AREA)
    k = np.zeros((5,5)); k[2,:] = 0.2
    small = cv2.filter2D(small,-1,k)
    _, enc = cv2.imencode(".jpg",small,[cv2.IMWRITE_JPEG_QUALITY,q])
    return cv2.resize(cv2.imdecode(enc,1),(224,224))
def embed(session, img):
    x = img.transpose(2,0,1)[None].astype("float32") / 255.0
    emb = session.run(None, {"image": x})[0][0]
    return emb / (np.linalg.norm(emb) + 1e-8) # L2 normalise
def generate_gallery(ref_dir, output="gallery.npy"):
    session = ort.InferenceSession(BACKBONE_PATH)
    gallery = {}
    for cls_dir in sorted(Path(ref_dir).iterdir()):
        embeds = [embed(session, degrade(cv2.imread(str(f))))
            for f in cls_dir.glob("*.jpg")]
        gallery[cls_dir.name] = np.stack(embeds) # (N, 128)
    print(f" {cls_dir.name}: {len(embeds)} embeddings")
    np.save(output, gallery)
    print(f"Gallery saved to {output}. No images transmitted.")
if __name__ == "__main__":
    generate_gallery("reference_images/")
```

5. Viewpoint-Aware Embedding — Approach 2

5.1 Why Naive Mean Prototype Fails at N=5 Across Angles

A front-on view of an MQ-9 Reaper shows an oval fuselage and a distinctive inverted-V tail. A side view shows a long, thin profile with a bulbous nose and a pusher propeller. A top-down view shows a wide wing planform with engine nacelles. A banking view shows the underside of the wing and landing gear pods. These four images share a common identity but have almost no overlapping local features. MobileNetV3's convolutional filters respond to texture and edges; the filter activations for a front view and a top-down view of the same object will be as different as activations for two entirely different aircraft.

When Approach 1 (naive mean) is applied, the prototype is the centroid of five diverse embedding vectors. This centroid is close to none of them individually. At runtime, a query from any particular angle will be closest to the corresponding support embedding, but the mean prototype averages that signal away. The result is suppressed similarity scores and increased miss rate, specifically at the angles that are least represented in the support set.

Naive mean at N=5 across angles is worse than N=1 same angle. If the client uploads five images from five different angles and the system averages them, it can perform worse than if the client uploaded a single representative side-on view. Approach 2 solves this by letting the model decide, at runtime, which of the five views is most relevant for the current query.

5.2 Attention-Weighted Aggregation Architecture

The attention head is a lightweight MLP that takes a query embedding and one support embedding as input and outputs a scalar relevance score. For a gallery with N support images, the attention head is applied N times (once per support view), and the N scores are normalised via softmax to produce attention weights. The adaptive prototype is the weighted sum of the N support embeddings. This prototype is query-conditioned: different queries from different angles produce different prototypes from the same gallery, dynamically emphasising the most relevant support view.

`src/model/attention_head.py` — AttentionHead class

```

import torch, torch.nn as nn, torch.nn.functional as F
class AttentionHead(nn.Module):
    """
    Query-conditioned attention over N support embeddings.
    Produces an adaptive prototype that emphasises the most
    viewpoint-relevant support images for a given query.
    Input: query (128,) – query crop embedding
    supports (N, 128) – all N support embeddings
    Output: adaptive_proto (128,)
    """
    def __init__(self, embed_dim=128, hidden=256):
        super().__init__()
        self.mlp = nn.Sequential(
            nn.Linear(4 * embed_dim, hidden),
            nn.ReLU(),
            nn.Dropout(0.1),
            nn.Linear(hidden, 64),
            nn.ReLU(),
            nn.Linear(64, 1)
        )
    def forward(self, query, supports):
        N = supports.shape[0]
        q = query.unsqueeze(0).expand(N, -1) # (N, 128)
        # relation features: concat, diff, hadamard
        feat = torch.cat([q, supports,
            q - supports,
            q * supports], dim=-1) # (N, 512)
        logits = self.mlp(feat).squeeze(-1) # (N,)
        weights = F.softmax(logits, dim=0) # (N,)
        return (weights.unsqueeze(-1) * supports).sum(0) # (128,)

```

5.3 Joint Training Strategy

The attention head is trained jointly with the backbone in Phase 2 (episodic training). It is not pre-trained separately. The intuition is that the backbone must learn to produce embeddings that the attention head can meaningfully score, and the attention head must learn to score embeddings that the backbone produces. They co-adapt.

Phase	What trains	Loss	Epochs
Phase 1: classification	Backbone only (attention head not involved)	Cross-entropy on class labels	20
Phase 2: episodic (N-way K-shot)	Backbone + attention head jointly	Cosine cross-entropy with temperature τ	40

During each episodic training step: sample a N-way K-shot episode. For each class, run the attention head using each query in the episode as the conditioning signal, producing a different adaptive prototype per query. Compute cosine similarity between each query and each class's adaptive prototype. Apply cross-entropy. Backpropagate through both the MLP and the backbone convolutional layers.

5.4 Runtime Inference Flow

runtime/inference.py — full identification pipeline on drone

```

# runtime/inference.py – runs on drone, uses both ONNX models
import onnxruntime as ort, numpy as np
backbone_sess = ort.InferenceSession("backbone_int8.onnx")
attn_sess = ort.InferenceSession("attention_head.onnx")
gallery = np.load("gallery.npy", allow_pickle=True).item()
THRESHOLD = 0.72 # tuned during Week 6 ablation
def identify(crop_bgr):
    """crop_bgr: (224, 224, 3) uint8 – from YOLO detection"""
    x = crop_bgr.transpose(2,0,1)[None].astype("float32") / 255.0
    query = backbone_sess.run(None, {"image":x})[0][0] # (128,)
    query /= np.linalg.norm(query) + 1e-8
    best_class, best_score = "unknown", 0.0
    for cls_name, supports in gallery.items():
        # attention head: query-conditioned prototype
        proto = attn_sess.run(None,
            {"query":query, "supports":supports})[0] # (128,)
        proto /= np.linalg.norm(proto) + 1e-8
        score = float(np.dot(query, proto))
        if score > best_score:
            best_class, best_score = cls_name, score
    if best_score >= THRESHOLD:
        return best_class, best_score
    return "unknown", best_score

```


6. Model Architecture

6.1 Backbone: MobileNetV3-Small

MobileNetV3-Small is selected on three grounds: TIDL operator compatibility (Conv, BN, Hardswish, SE blocks are all supported), lightweight parameter count (~2.5 M parameters, ~2.8 MB FP32 weights), and strong ImageNet pre-training that transfers well to aerial object recognition. The classification head is replaced with a projection layer mapping the 576-dim pool output to a 128-dim L2-normalised embedding.

src/model/backbone.py

```
import torchvision.models as M
import torch.nn as nn
class EmbeddingBackbone(nn.Module):
    def __init__(self, embed_dim=128):
        super().__init__()
        base = M.mobilenet_v3_small(pretrained=True)
        self.features = base.features
        self.pool = base.avgpool
        self.proj = nn.Sequential(
            nn.Linear(576, embed_dim),
            nn.LayerNorm(embed_dim)
        )
    def forward(self, x):
        feat = self.pool(self.features(x)).flatten(1)
        emb = self.proj(feat)
        return F.normalize(emb, dim=-1) # L2 unit sphere
```

6.2 Attention Head: Lightweight MLP

Layer	Input dim	Output dim	Activation
Linear 1	512 (4 × 128)	256	ReLU + Dropout(0.1)
Linear 2	256	64	ReLU
Linear 3	64	1 (scalar score)	None (raw logit)
Softmax	N logits	N weights	Normalises over support set

Total parameters: $512 \times 256 + 256 \times 64 + 64 \times 1 = 131,136 + 16,384 + 64 = \sim 147,584$. This is negligible compared to the backbone. Inference time for N=5 is approximately 0.1 ms on a modern ARM Cortex-A core.

6.3 ONNX Export for Both Models

scripts/export_onnx.py — all three exports

```

# scripts/export_onnx.py
# 1. Export backbone (FP32)
backbone.eval()
dummy_img = torch.randn(1, 3, 224, 224)
torch.onnx.export(backbone, dummy_img, "backbone_fp32.onnx",
opset_version=11,
input_names=["image"], output_names=["embedding"])
# 2. INT8 quantization of backbone
from onnxruntime.quantization import quantize_dynamic, QuantType
quantize_dynamic("backbone_fp32.onnx", "backbone_int8.onnx",
weight_type=QuantType.QInt8)
# 3. Export attention head (FP32 only, stays small)
attn_head.eval()
q_dummy = torch.randn(128)
s_dummy = torch.randn(5, 128) # N=5 at export time
torch.onnx.export(attn_head, (q_dummy, s_dummy),
"attention_head.onnx",
opset_version=11,
input_names=["query","supports"],
output_names=["adaptive_prototype"],
dynamic_axes={"supports": {0: "n_shots"}})
# dynamic_axes allows N to vary at runtime (1,3,5,10...)

```

7. Dataset Strategy — UAV Domain

7.1 Available Public Datasets

The target class is UAVs only — consumer, commercial, and military unmanned aerial vehicles. Planes, helicopters, and ground vehicles are not in scope for this TFG iteration. The datasets below are selected specifically for UAV content against sky backgrounds, which is the domain the backbone must learn.

Dataset	Content	Size	Source	Use
Drone-vs-Bird	Consumer and racing drones vs birds against sky; video sequences	~77K frames	IEEE Open Access	Primary
Anti-UAV (CVPR 2021)	Military + consumer UAVs; thermal + RGB; varied distances	~300 sequences	GitHub release	Primary
MAV-VID	Micro aerial vehicles in flight; drone-to-drone perspective	~40K frames	Benchmark site	Pre-training
DUT Anti-UAV	UAV sequences, fixed camera; diverse backgrounds	~10K frames	DUT, open access	Pre-training
Roboflow — drone detection (15+ datasets)	Consumer and commercial drones; annotated bounding boxes; varied angles	~15K images total	universe.roboflow.com	Pre-training
FL-Drones	Small UAVs in sky; Florida dataset; low-altitude captures	~3K images	University release	Secondary

7.2 UAV Reference Images for the Ablation Study

The ablation study needs N-shot reference images of specific UAV models from multiple angles. Military datasets such as DVIDS (dvidshub.net, public domain US DoD media) contain TB2 Bayraktar and other military UAV photographs. For the TFG proof of concept, consumer and commercial UAVs are the most practical choice: DJI Phantom 4, DJI Mavic 3, Autel Evo 2, Parrot Anafi, and Skydio 2 all have hundreds of clean photographs from varied angles freely available from manufacturer press kits, review channels, and community forums — no license issues for research use. These become the enrolled classes in the ablation study.

The key challenge at this scale class (2×1 m, consumer UAV size range) is **intra-class confusion**: a DJI Mavic 3 and a DJI Air 2S are both foldable quadrotors with nearly identical silhouettes at 75 m. The ablation study must include at least two same-category negative test cases — enrol one quadrotor and query with a different quadrotor of the same approximate size. The result directly characterises the system's discrimination limit, which is the most important single number in the final Embention report.

Synthetic data (Blender) supplements real photographs for angles that are difficult to capture: directly overhead, directly below, and nose-on. A 3D model rendered at 360° azimuth × 5 elevation angles × 3

lighting conditions produces 5,400 images per UAV class. These are used during episodic training to expose the attention head to the full viewpoint distribution, not during enrollment (where only real client photos are used).

7.3 Augmentation Pipeline for UAV Domain

Augmentation	What it simulates	Parameters
Sky background replacement	Clear, overcast, dawn, sunset sky conditions	20 background pool, random crop
Resolution degradation (optional)	YOLO crop at 50–100 m operational distance	target_px ~ U(46, 143); prob=0.7
Horizontal motion blur	Forward flight of camera drone	Kernel size 3–9 px
Sun glare	High-angle sun; common in UAV operations	Gaussian highlight, random position
HSV jitter	Atmospheric haze, different lighting conditions	H±15, S±30%, V±20%
Banking rotation	UAV banking left/right during flight	Random rotation ±30°
Partial occlusion	Partial cloud cover; edge-of-frame crops	Random rectangle mask, 10–30%

8. Quantization Pipeline

8.1 Why INT8 Matters for Edge Hardware

Although the specific target hardware is deferred, INT8 quantization is a universal requirement for embedded AI deployment. INT8 reduces model size by 4× (from ~2.8 MB to ~0.7 MB for the backbone), reduces memory bandwidth by 4×, and on hardware with dedicated INT8 MAC units (like the TDA4VH-Q1's MMAV2) can provide 4–8× throughput improvement over FP32. The accuracy loss with post-training quantization is typically under 2% for well-trained models on their representative data distribution.

8.2 Post-Training Quantization Flow

scripts/quantize.py — both dynamic and static INT8

```
# scripts/quantize.py
from onnxruntime.quantization import (
    quantize_dynamic, quantize_static,
    CalibrationDataReader, QuantType
)
import numpy as np

# Option A: Dynamic quantization (weights only, no calibration data)
# Fast, works immediately, slight accuracy penalty vs static
quantize_dynamic(
    model_input="backbone_fp32.onnx",
    model_output="backbone_int8_dynamic.onnx",
    weight_type=QuantType.QInt8
)

# Option B: Static quantization (activations also quantized)
# Better accuracy, requires ~100 representative calibration crops
class DroneCalibReader(CalibrationDataReader):
    def __init__(self, crops):
        self.crops = iter(crops)
    def get_next(self):
        try: return {"image": next(self.crops)}
        except StopIteration: return None
# Load 100 representative drone crops (from validation set)
calib_crops = load_calib_crops("data/calib/", n=100)
quantize_static(
    model_input="backbone_fp32.onnx",
    model_output="backbone_int8_static.onnx",
    calibration_data_reader=DroneCalibReader(calib_crops)
)
```

8.3 What Gets Quantized and What Does Not

Model / Data	Quantized?	Reason
backbone_fp32.onnx → backbone_int8.onnx	Yes (INT8 static)	Heaviest model; biggest benefit; accuracy loss acceptable
attention_head.onnx	No (stays FP32)	Only 147K params; quantization benefit negligible; precision matters for softmax

gallery.npy embeddings	No (stays FP32)	128 floats per class; 2.5 KB total; memory cost negligible
------------------------	-----------------	--

8.4 Hito 3 Deliverable Specification

The Hito 3 submission includes three ONNX files, one comparison table, and a benchmark script. Embention's hardware team takes the ONNX files and feeds them into their target platform's compiler (TIDL, TensorRT, or similar). The TFG does not perform platform-specific compilation — that is the handoff boundary.

File	Size estimate	Purpose
backbone_fp32.onnx	~2.8 MB	Reference model; used by client app for enrollment
backbone_int8.onnx	~0.7 MB	Deployment model for drone runtime; INT8 static quantized
attention_head.onnx	~0.6 MB	Runtime attention aggregation; dynamic N via dynamic_axes
accuracy_latency_table.csv	< 1 KB	FP32 vs INT8 accuracy (5-way 5-shot) and x86 CPU latency
scripts/benchmark_onnx.py	N/A	Reproducible benchmarking script; runs on any x86 Linux host

9. N-shot Ablation — A Product Question

9.1 Reframing: This is Product Specification Research

The N-shot ablation in this TFG is not purely academic. Its output answers a concrete product question that Embention must be able to communicate to clients: **how many photographs of a target do you need to upload, and what kinds of photographs?** The answer becomes the client onboarding documentation. If the answer is "you need at least 5 images covering front, side, top-down, and banking angles to achieve 90% identification accuracy at 75 m", that is a product specification backed by your experimental data.

The ablation study is the core scientific contribution of this TFG. It determines the minimum viable enrollment protocol, quantifies the cost of using fewer or worse images, and provides Embention with data to set client expectations before deployment.

9.2 Test Matrix: N and Image Type

The ablation crosses two variables: the number of support images N, and the diversity of viewpoints in those images. Each cell in the matrix is evaluated over 1,000 random episodes from the held-out test set.

Image type / N	N=1	N=3	N=5	N=10	N=20
Same angle (worst case)	■	■	■	■	■
2 angles (front + side)	N/A	■	■	■	■
Diverse angles (recommended)	N/A	■	■	■	■
Mixed quality (some degraded refs)	■	■	■	■	■

■ = cells to be filled by ablation experiments in Week 6.

9.3 Expected Output: Practical Recommendation

The primary output is an accuracy vs N curve, plotted separately for each image type category. Secondary outputs are: the confidence threshold value that minimises false positives in the test set, and the performance degradation when reference image quality is deliberately reduced (simulating clients who only have low-quality photos).

The expected practical recommendation, to be confirmed or revised by experimental results, is: *"We recommend uploading a minimum of 5 reference images covering at least three distinct viewing angles (front-on, side-on, and top-down or banking). Using more than 10 images provides diminishing returns. Single-angle enrollment is supported but reduces identification accuracy by approximately X%."* The X is what your experiments will measure.

10. Risks and Mitigations

Risk	Prob	Impact	Mitigation
33-pixel object: backbone fails on degraded input	High	High	Resolution-matched enrollment (Section 2.3). Train with random degradation. Evaluate specifically on simulated 100 m crops. Fix in Week 2 augmentation pipeline.
Attention head overfits to training viewpoints	Medium	High	Use diverse synthetic angles during Phase 2 training. Apply aggressive viewpoint augmentation. Evaluate on held-out angles not seen during training.
Client uploads all images from same angle	High	Medium	System degrades gracefully: same-angle N=5 is still better than N=1. Document in client guidance. Add a diversity warning to the client app output.
Military UAV dataset scarcity	High	Medium	Use DVIDS (public domain US DoD media) for reference images. Supplement with synthetic Blender renders at varied angles. Frame TFG as proof-of-concept on surrogate civilian drone classes.
Attention head ONNX export fails on dynamic N	Medium	Medium	Export with dynamic_axes on the supports dimension. Test N=1,3,5,10 all with the same exported model in Week 9. Fallback: export N=5 fixed-size version if dynamic export fails.
INT8 quantization causes >3% accuracy drop	Low	Medium	Switch to static quantization with 100 calibration crops. Try mixed INT8/FP16 for sensitive layers. Document the drop and let Embention decide the acceptable threshold.
Sky background confuses detector (false positives on birds/clouds)	Medium	Low	YOLO pre-trained on COCO already distinguishes birds from aircraft. Fine-tune on Drone-vs-Bird dataset in Week 2. Add confidence threshold on YOLO detections before passing to identification module.

11. Open Questions for Embention

Question	Needed by	Reason
Which Processor SDK version is deployed on the KAI platform? (Linux: 11-01-00-03 confirmed; verify for ONNX RT build.)	Week 1 (confirmed)	SDK 11.01 confirmed → use edgeai-tidl-tools tag 11_01_xx. No blocker remains for Week 1.
What is the camera sensor resolution and FOV on the target drone?	Week 2 (urgent)	Determines precise pixel count at 50–100 m. Changes degradation parameters in the enrollment pipeline. Must be locked before dataset prep.
How is gallery.npy transferred to the drone pre-flight? (USB, SD card, local WiFi, RF link)	Week 4	Determines the encryption and transfer protocol to document in the privacy architecture section of the final report.
Is J784S4EVM available for SSH access during Weeks 9–10?	Week 9	Required for real latency numbers. Edge AI Studio gives approximate estimates if hardware is unavailable.
Does KAI consume backbone_int8.onnx directly, or does Embention's hardware team perform a second TIDL compilation step?	Week 10	Determines whether the ONNX deliverable is final or an intermediate. Changes the demo integration stub.

12. Development Roadmap — 12 Weeks

The roadmap is structured across three months. Month 1 builds the foundational infrastructure: hardware validation, aerial datasets, the degradation pipeline, and Phase 1 backbone training. Month 2 implements the attention head, runs the N-shot ablation, finalises the client app, and characterises robustness. Month 3 exports all ONNX models, quantizes the backbone, builds the demo pipeline, and prepares the final deliverables.

Month 1 — Foundation

Week 1 Environment & TIDL Smoketest

Milestone: Confirmed backbone compiles on TIDL. Camera FOV confirmed.

- Monday: Upload MobileNetV3-Small ONNX to TI Edge AI Studio. Confirm Hardswish compiles on am69a. If it fails, switch to MobileNetV2 today — max 2 hrs debugging.
- Tuesday: Request camera sensor spec (resolution and FOV) from Embention. This unlocks precise pixel-count calculation for the degradation pipeline. Compute target_px values for 50, 75, 100 m and document them.
- Wednesday: Literature review — Snell et al. (2017) Prototypical Networks; Sung et al. (2018) Relation Networks (for attention head background). Write 2-page architecture justification document for Hito 1.
- Thursday: Set up WSL2 (Ubuntu 22.04). Clone edgeai-tidl-tools, checkout tag 11_01_xx (use git tag | grep 11_01 to find the latest). Set SOC=am69a. Run setup.sh. Verify example model compiles.
- Friday: Scaffold the project repository. Set up the five main source directories (src/model, src/data, src/inference, scripts, notebooks). Write stub versions of generate_gallery.py and demo.py. Confirm YOLO integration stub produces fake embeddings end-to-end.

Deliverable: backbone.onnx compiling on TIDL + repo scaffold + architecture justification + pixel-count table

Week 2 Sky Domain Datasets & Augmentation Pipeline

Milestone: Training-ready aerial dataset with degradation augmentation

- Download Drone-vs-Bird, Anti-UAV (CVPR 2021), and VisDrone2019-DET. Extract individual frame images from video sequences. Verify licenses (IEEE Open Access, challenge data terms).
- Download 200+ MQ-9 / TB2 / drone images from DVIDS (dvidshub.net). These become the few-shot reference images for the ablation study. All are public domain.
- Implement degrade_reference() (Section 2.3) and integrate it into the Albumentations augmentation pipeline. Test: run the same image through degradation at 50 m / 75 m / 100 m, visually inspect output, confirm backbone receives realistic-looking inputs.
- Implement additional sky-domain augmentations: sky background replacement (20 background images pool), sun glare overlay, banking rotation $\pm 30^\circ$, partial occlusion mask. Integrate all into a single AerialAugment class.
- Build the unified train/val/test split (80/10/10 stratified by class). Visualise 50 augmented samples. Log class distribution bar chart. Confirm no class leakage across splits.

Deliverable: AerialAugment class + unified dataset + degrade_reference() + class distribution notebook

Week 3**Phase 1 Training —
Classification Baseline***Milestone: Backbone achieves >80% top-1 on aerial classes*

- Implement EmbeddingBackbone (Section 6.1): MobileNetV3-Small features + avgpool + Linear(576, 128) + LayerNorm. Replace classification head. Apply L2 normalisation at output.
- Phase 1 training: AdamW (lr=1e-3), cosine LR decay to 1e-5, 20 epochs, batch 64. Apply `degrade_reference()` to every training image (random target_px 33–66). All images are degraded — no clean images during training.
- Log: top-1 accuracy every epoch, confusion matrix every 5 epochs. Identify worst class pairs (front-on drone A vs front-on drone B). These drive the attention head design in Month 2.
- Generate t-SNE visualisation of the 128-dim embedding space at epoch 20. Classes must form separable clusters. If they do not, diagnose backbone training before proceeding — proceeding with non-separable embeddings will make Phase 2 fail.
- Save the best checkpoint by validation accuracy. This is the starting point for Phase 2 fine-tuning.

Deliverable: Trained backbone checkpoint + confusion matrix + t-SNE plot**Week 4****Baseline Evaluation & Hito 1
Report***Milestone: Hito 1 deliverable submitted*

- Evaluate Phase 1 backbone on test set: top-1 accuracy, macro F1, per-class ROC curves. Record these as the "supervised baseline" for comparison in the final report.
- Run few-shot evaluation using mean prototype (Approach 1) at N=1 and N=5 to establish the naive baseline before adding the attention head. This gives the before/after comparison for Section 5.
- Verify gallery generation pipeline end-to-end: `generate_gallery.py` with `backbone_fp32.onnx` produces `gallery.npy`, load it, run `identify()` on test crops, confirm similarity scores are in expected range.
- Write Hito 1 report: state of the art review (Prototypical Networks, attention-based few-shot, aerial object detection), architecture justification, dataset analysis (class distribution, degradation examples), baseline results table.
- Review Hito 1 against TFG success criteria: viability, trade-off analysis, code quality, documentation. Identify any gaps before Month 2.

Deliverable: Hito 1 report draft + naive baseline accuracy table + gallery pipeline verified**Month 2 — Attention Module & Ablation**

Week 5**Attention Head
Implementation & Joint
Training***Milestone: Attention head training loop running, first accuracy numbers above naive baseline*

- Implement AttentionHead class (Section 5.2): Linear(512,256) + ReLU + Dropout(0.1) + Linear(256,64) + ReLU + Linear(64,1). Verify forward pass shapes: input (N,512) → output (128,) adaptive prototype.
- Implement episodic training loop: for each episode, sample N-way K-shot tasks. For each class, for each query in the episode, compute attention-weighted adaptive prototype. Loss: cosine cross-entropy with learnable temperature.
- Phase 2 training: load Phase 1 backbone checkpoint. Add attention head. Train jointly for 40 epochs with lower lr=1e-4 for backbone and lr=5e-4 for attention head (different learning rates prevent destroying Phase 1 features).
- First evaluation: 5-way 1-shot and 5-way 5-shot with same-angle and diverse-angle support sets. Compare: naive mean prototype (Approach 1) vs attention-weighted (Approach 2). Document delta.
- Visualise attention weights: for a test query, which of the 5 support images does the attention head weight most? Does it upweight the support image from the matching angle? Plot attention weight distributions.

Deliverable: AttentionHead checkpoint + Phase 2 accuracy table + attention visualisation plots**Week 6 N-shot Ablation Study***Milestone: Full ablation matrix complete; practical N recommendation written*

- Fill the ablation matrix (Section 9.2): test 5 values of N (1,3,5,10,20) × 4 image type categories × 1,000 episodes each. This is 20 experiment runs. Automate with scripts/run_ablation.py.
- For each cell, record: mean accuracy, 95% confidence interval, false positive rate, false negative rate at threshold 0.72. Use the same threshold across all cells for fair comparison.
- Backbone comparison: MobileNetV3-Small vs MobileNetV2 on the 5-way 5-shot diverse-angle condition. Select the winner based on accuracy/latency trade-off. Freeze the backbone choice.
- Determine the practical recommendation: find the knee of the accuracy vs N curve (point of diminishing returns). This is the minimum recommended N for client enrollment.
- Write the ablation results section for the final report. Include accuracy vs N plot, attention weight analysis, and the practical recommendation in plain language.

Deliverable: Full ablation matrix + accuracy vs N plot + written practical recommendation

Week 7 Client App & Gallery Script

Milestone: *generate_gallery.py* fully tested; client app privacy guarantee demonstrable

- Finalise `generate_gallery.py` (Section 4.4): degradation pipeline, backbone ONNX inference, L2 normalisation, `gallery.npy` output. Test with 5 MQ-9 images from DVIDS at $N=1,3,5$.
- Verify privacy guarantee: run Wireshark while executing `generate_gallery.py`. Confirm zero outbound packets. Document this in the privacy section of the report with a screenshot.
- Add CLI argument parsing: `--model`, `--input_dir`, `--output`, `--distance` (50/75/100 m $\rightarrow$ sets `target_px` automatically). Add `--verbose` flag that prints per-image embedding statistics.
- Add a diversity check warning: if all N support embeddings have pairwise cosine similarity > 0.95 , print "WARNING: all reference images appear to be from the same angle. Consider adding views from different angles for better performance."
- Test end-to-end: generate gallery from 5 DVIDS images $\rightarrow$ load gallery on simulated runtime $\rightarrow$ run `identify()` on 20 test crops $\rightarrow$ confirm correct classifications.

Deliverable: `generate_gallery.py` (final version) + privacy demo (Wireshark screenshot) + diversity warning

Week 8 Robustness Testing

Milestone: *Failure mode analysis complete; robustness section written for final report*

- Degrade reference images beyond training distribution: blur (sigma 4,8), JPEG $q=25$, partial occlusion 40%, extreme sun glare. Measure accuracy at $N=5$ diverse angles under each condition.
- Degrade query images: fog simulation (additive white), night ($\gamma=0.2$), rain overlay, extreme motion blur. Measure identification accuracy at each condition.
- Test false positive rate: enrol one class, query with 50 images from completely different aerial objects (birds, helicopters, different drone models). Count false positives above threshold.
- Test cross-distance generalisation: enrol with 75 m degradation, query with 50 m and 100 m degradation (different from enrollment). Measure accuracy drop. This tells us whether the degradation level must be matched between enrollment and runtime.
- Write limitations section: list conditions where the system fails, recommended mitigations, and known open problems. This section must be present in the final report for academic credibility.

Deliverable: Robustness tables + false positive rate analysis + limitations section draft

Month 3 — Optimisation & Demo

Week 9 ONNX Export & Quantization*Milestone: All three ONNX files exported, INT8 accuracy delta documented*

- Export backbone_fp32.onnx with opset=11. Verify on onnxruntime CPU: output matches PyTorch within 1e-4. Run existing ablation test cases through ONNX model — results must match checkpoint results.
- Export attention_head.onnx with dynamic N (dynamic_axes on supports dimension). Test N=1,3,5,10 all with the same exported model. Verify output matches PyTorch.
- INT8 dynamic quantization: backbone_fp32 → backbone_int8_dynamic.onnx. Measure accuracy delta on 5-way 5-shot test set.
- INT8 static quantization with calibration data: backbone_fp32 → backbone_int8_static.onnx. Use 100 representative degraded drone crops as calibration set. Measure accuracy delta. Compare static vs dynamic.
- Latency profiling on x86: time backbone_fp32 + attention_head per inference call (1 crop, N=5). Time backbone_int8_static + attention_head. Report median over 1000 calls. Note: x86 latency is a relative baseline, not the target hardware number.

Deliverable: backbone_fp32.onnx + backbone_int8.onnx + attention_head.onnx + accuracy/latency table**Week 10 End-to-End Demo Pipeline***Milestone: demo.py runs on aerial video and correctly identifies enrolled target*

- Integrate pre-trained YOLOv8n as upstream detector. Configure for aerial objects: confidence threshold 0.4, NMS IoU 0.5. Test on drone footage from MAV-VID dataset.
- Write scripts/demo.py: video/image input → YOLOv8n detection → crop (224x224) → backbone_int8.onnx embedding → attention_head.onnx adaptive prototype → cosine similarity vs gallery → annotated output with class label + score.
- Test privacy integration: generate gallery using generate_gallery.py (5 DVIDS images of target), load gallery.npy in demo.py, confirm correct identification on test footage.
- Add confidence display: draw bounding box in green if similarity > 0.72, yellow if 0.55–0.72, skip if < 0.55. Display class name + score on the bounding box.
- Record a 60-second demo video: enrol a target class (e.g. consumer DJI drone from DVIDS), run demo.py on footage containing that drone plus other aerial objects (birds, planes), show correct identification and rejection.

Deliverable: scripts/demo.py + 60-second demo video + generate_gallery.py integration test

Week 11 **Code Cleanup & Documentation**

Milestone: Repository fully reproducible from clean clone

- Enforce final repository structure: src/model/ (backbone.py, attention_head.py), src/data/ (dataset.py, augment.py), src/inference/ (identify.py), scripts/ (train.py, export_onnx.py, quantize.py, run_ablation.py, benchmark_onnx.py), client_app/ (generate_gallery.py), notebooks/, exports/, checkpoints/.
- Write README.md: installation (pip install -r requirements.txt), dataset setup commands, training command, export command, quantization command, gallery generation, demo usage. Another engineer must be able to reproduce all results from README alone.
- Add docstrings and type hints to all public functions. Remove dead code and debug prints. Enforce Black formatting.
- Verify reproducibility: fresh conda environment, pip install, single training command. Results must match documented numbers within $\pm 0.5\%$ accuracy.
- Write final LaTeX memory draft: abstract, introduction, related work, methodology (Sections 2–6 of this document), experiments (ablation tables, robustness analysis), limitations, conclusions. Allocate the most pages to the experimental results section.

Deliverable: Clean repository + requirements.txt + README.md + LaTeX memory draft

Week 12 **Final Report & Embention Presentation**

Milestone: TFG submitted; Embention presentation delivered

- Finalise LaTeX memory: ensure all four success criteria (Section 1.3) are explicitly addressed. Check that the legal framing (Executive Summary E.2) appears in the introduction.
- Prepare Embention presentation (12–15 slides): problem statement, pipeline diagram, privacy architecture (Option A flow), attention head motivation, ablation results (accuracy vs N plot), demo video, INT8 vs FP32 comparison, limitations, next steps (GUI, hardware compilation, live field test).
- Prepare live demo for the presentation: laptop + pre-recorded demo video as backup. Test the full pipeline end-to-end on the presentation laptop the day before.
- Verify against all TFG success criteria: viability demo (Section 1.3, criterion 1), trade-off analysis (criterion 2), privacy guarantee (criterion 3), ONNX exports (criterion 4), code quality (criterion 5).
- Submit TFG memory and repository link. Archive the three ONNX files, the demo video, and the gallery.npy used in the demo in the repository releases.

Deliverable: TFG submission + Embention slides + demo video + ONNX file archive